

From Pipeline to Agent: Voyage Cancellations Agent

The Next Challenge

In Session 1 you built a Travel Policy Assistant that can answer questions about Voyage's refund and cancellation rules. The support team is pleased — but they have a new request.

They don't just want the assistant to *answer* questions. They want it to *act*: look up a booking, check whether it is eligible for a refund, and process the cancellation if it is. The current pipeline cannot do any of this — it can only retrieve text and generate a response.

Our Task

A RAG pipeline is designed to answer questions, but an agent is designed to take actions — and when a system needs to make decisions, call APIs, and complete workflows, we need an agent rather than a static retrieval pipeline. Our task today is to convert our RAG pipeline into an agentic flow.

Smolagents vs LangGraph: Prototype and System Design

There are multiple ways to build agents in Python, and the choice often depends on the level of structure and control you need.

Smolagents is designed to make it very easy to give a model access to tools. With minimal setup, you can prototype task-oriented agents quickly. It works well when you want to experiment with tool use, explore automation ideas, or build lightweight workflows without managing the underlying execution loop yourself.

LangGraph, by contrast, makes the reasoning loop explicit. It allows you to define how tools are orchestrated, how memory is handled, and how execution progresses from step to step. This makes it better suited for more structured systems, especially when you want visibility into intermediate reasoning or need tighter control over behavior.

In practice, smolagents is often a strong choice for rapid prototyping, while LangGraph provides more flexibility for building structured, multi-step workflows. As systems grow in complexity — especially when retrieval, memory, and multi-tool coordination are involved — having explicit control over orchestration becomes increasingly important.

```
!pip install --quiet langchain-core==0.3.59 langgraph==0.4.3 langchain-openai==0.3.16 langchain-experimental==0.3.4 langgraph-supervisor==0.0.21
```

```
[notice] A new release of pip is available: 25.2 -> 26.1.1
```

```
[notice] To update, run: pip install --upgrade pip
```

```
import os
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_text_splitters import MarkdownHeaderTextSplitter
from langchain_core.tools import tool
from langchain_core.documents import Document
from langgraph.prebuilt import create_react_agent
```

```
openai_key = os.environ["OPENAI"]
```

Rebuild the Policy Retriever

We rebuild the vector store from Session 1 so this notebook runs standalone. The agent will call this via a `lookup_policy` tool.

```
# Import our policy text
with open("travel_policy.txt", "r") as f:
    raw_text = f.read()

# Set up the Chroma DB
headers = [("#", "Title"), ("##", "Section"), ("###", "Subsection")]

chunks =
MarkdownHeaderTextSplitter(headers_to_split_on=headers).split_text(raw_text)

vector_db = Chroma.from_documents(
    documents=chunks,
    embedding=OpenAIEmbeddings(model="text-embedding-3-small",
                                openai_api_key=openai_key),
    collection_metadata={"hnsw:space": "cosine"}
)

# Configure the retriever
retriever = vector_db.as_retriever(
    search_type = "similarity",
    search_kwargs={"k": 3})
```

Build Your First Tool

Tools are standard Python functions decorated with `@tool`. The **docstring** is what the model reads to decide when to call the tool — a clear, specific docstring leads to better decisions than a vague one.

We start with a single tool that wraps the Session 1 retriever.

Tool 1: The Policy Lookup This tool gives the Agent access to our RAG retriever. We write a broad docstring so the Agent knows it can use this for any policy question.

```
## Create a tool to retrieve the relevant context
@tool
def lookup_policy(query:str)-> str:
    """
    Consult the official Voyage Cancellation Policy
    Use this tool to verify refund rules or check cancellation fees.
    """
    docs = retriever.invoke(query)
    return "\n\n".join([d.page_content for d in docs])
```

```
# Define the tool set
tools=[lookup_policy]
```

Wire the Agent

We now combine the language model and our tools into a single reasoning loop. The agent follows the ReAct pattern (Reason + Act): it thinks about what to do, selects a tool if needed, observes the result, and continues reasoning until it can produce a final answer. Unlike a traditional function call, this loop allows the model to dynamically decide the next step based on new information it receives from tools.

Importantly, the agent is not simply calling a tool once. It can call multiple tools in sequence, using the output of one step to inform the next. This ability to reason over intermediate results is what makes agents powerful for workflow automation.

```
llm = ChatOpenAI(model="gpt-4o-mini",
                  openai_api_key=openai_key,
                  temperature=0)
```

```
# These instructions act as the strict rules of engagement for the agent.
system_prompt = """
### ROLE
You are the "Voyage Cancellations Agent." Your primary job is to process flight
cancellation requests accurately and securely.

### WORKFLOW & CONSTRAINTS
1. **MANDATORY VERIFICATION:** You must NEVER cancel a ticket without first
verifying the refund policy for that specific ticket class. Use the `lookup_policy`
tool to check the rules.
2. **NON-REFUNDABLE TICKETS:** If the policy states a ticket is non-refundable, you
must politely refuse the request and explain why.
3. **REFUNDABLE TICKETS:** If the policy permits a refund (and any conditions like
'24 hours prior' are met), you should confirm that the customer is able to cancel
their ticket.

### TONE
Professional, objective, and direct. Do not apologize for enforcing company policy.
"""
```

```
# Set up the agent executor
agent_executor = create_react_agent(
    llm,
    tools,
    prompt=system_prompt
)
```

Test the Policy Agent

With `verbose=True` you will see every Thought, Action, and Observation printed as the agent runs. Watch the reasoning chain — this is the ReAct loop in action.

```
query = "I want to cancel my Saver Ticket #12345."
```

```
# Run the executor and stream each step
for step in agent_executor.stream(
    {"messages": [("user", query)]},
    stream_mode="updates",
):
    print("\n--- STEP ---")
    print(step)
```

--- STEP ---

```
{'agent': {'messages': [AIMessage(content='', additional_kwargs={'tool_calls':
[{'id': 'call_HetthA1U7Ns7yWuCxt2M8mo', 'function': {'arguments':
{'query': 'Saver Ticket'}}, {'name': 'lookup_policy'}, {'type': 'function'}]},
'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 15,
'prompt_tokens': 237, 'total_tokens': 252, 'completion_tokens_details':
{'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0,
'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0,
'cached_tokens': 0}}, 'model_name': 'gpt-4o-mini-2024-07-18',
'system_fingerprint': 'fp_3620f71984', 'id': 'chatcmpl-
Df7U2rEbrfj5iAShOF4d3ke5pDGLY', 'service_tier': 'default', 'finish_reason':
'tool_calls', 'logprobs': None}, id='run--0ff158c8-632d-4417-bbea-fb449a624165-0',
tool_calls=[{'name': 'lookup_policy', 'args': {'query': 'Saver Ticket'}, 'id':
'call_HetthA1U7Ns7yWuCxt2M8mo', 'type': 'tool_call'}], usage_metadata=
{'input_tokens': 237, 'output_tokens': 15, 'total_tokens': 252,
'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details':
{'audio': 0, 'reasoning': 0}})]}}
```

--- STEP ---

```
{'tools': {'messages': [ToolMessage(content='* **Refund Status:** Non-
refundable.\n* **Changes:** Not permitted.\n* **Exceptions:** If the airline
cancels the flight, a full refund will be issued.\n* **Note:** Saver fares do not
include checked baggage.\n\n* **Refund Status:** Fully refundable up to 2 hours
before departure.\n* **No-Show Policy:** If you fail to check in, the ticket value
is converted to Voyage Credits valid for 1 year.\n\n*Note: This section is for
historical reference and applies only to bookings made before Jan 1, 2024.*\n*
**All Tiers:** Previously, all tickets were 100% refundable if cancelled within 24
hours of booking, regardless of departure time. This has been superseded by the
```

```
query = "I want to cancel my Standard Ticket #9999. It is for next week."
```

```
query = "I want to cancel my Main Cabin Ticket #9999. It is for next week."
```

```
for step in agent_executor.stream(
    {"messages": [("user", query)]},
    stream_mode="updates",
):
    print("\n--- STEP ---")
    print(step)
```

--- STEP ---

```
{'agent': {'messages': [AIMessage(content='', additional_kwargs={'tool_calls':
[{'id': 'call_9wz2dtUJNrSIS1JugCcadfcI', 'function': {'arguments': '{"query": "Main
Cabin Ticket"}', 'name': 'lookup_policy'}, 'type': 'function'}], 'refusal': None},
response_metadata={'token_usage': {'completion_tokens': 16, 'prompt_tokens': 244,
'total_tokens': 260, 'completion_tokens_details': {'accepted_prediction_tokens':
0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0},
'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_name':
'gpt-4o-mini-2024-07-18', 'system_fingerprint': 'fp_3620f71984', 'id': 'chatcmpl-
Df7c9r7pKyBJ0wKs6R8dyDqaFHoVy', 'service_tier': 'default', 'finish_reason':
'tool_calls', 'logprobs': None}, id='run--939a89f5-30bc-4170-9dde-1610e7feace4-0',
tool_calls=[{'name': 'lookup_policy', 'args': {'query': 'Main Cabin Ticket'},
'id': 'call_9wz2dtUJNrSIS1JugCcadfcI', 'type': 'tool_call'}], usage_metadata=
{'input_tokens': 244, 'output_tokens': 16, 'total_tokens': 260,
'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details':
{'audio': 0, 'reasoning': 0}}]}
```

--- STEP ---

```
{'tools': {'messages': [ToolMessage(content='* **Refund Status:** Fully refundable
up to 2 hours before departure.\n* **No-Show Policy:** If you fail to check in,
the ticket value is converted to Voyage Credits valid for 1 year.\n\n*
**Cancellation Window:**\n* **More than 48 hours before departure:** 100%
refundable to original payment method.\n* **Within 48 hours of departure:** 50%
cancellation fee applies. The remaining 50% is issued as Voyage Credits.\n*
**Within 4 hours of departure:** Non-refundable.\n* **Changes:** One date change
is permitted per booking, subject to fare difference.\n\n* **Economy:** $50 per
checked bag.\n* **Business:** 2 bags included.\n* **Special Items:** Surfboards
and Skis incur a non-refundable $75 fee per leg.', name='lookup_policy',
```

Expanding the Toolkit

Policy lookup alone is not enough. To process a cancellation the agent also needs to:

1. **Look up the booking** to find out the ticket type.
2. **Cancel the ticket** once the policy confirms it is eligible.

We add two more tools. Notice how the docstrings guide the agent's decision on *when* to use each one.

```
# Create a second tool
@tool
def cancel_ticket(ticket_id: str) -> str:
    """
    Cancels a flight booking immediately.
    WARNING: You must use the 'lookup_policy' tool to verify if the the ticket is
    refundable BEFORE calling this tool. Do not cancel a non-refundable ticket.
    """
    return f"Success: Ticket {ticket_id}' has been cancelled"
```

```
# Update the tool set
tools = [lookup_policy, cancel_ticket]
```

Wire the Full Agent

We now rebuild the agent with the new workflow.

```
system_prompt = """
### ROLE
You are the "Voyage Cancellations Agent." Your primary job is to process flight
cancellation requests accurately and securely.

### WORKFLOW & CONSTRAINTS
1. **MANDATORY VERIFICATION:** You must NEVER cancel a ticket without first
verifying the refund policy for that specific ticket class. Use the `lookup_policy`
tool to check the rules.
2. **NON-REFUNDABLE TICKETS:** If the policy states a ticket is non-refundable, you
must politely refuse the request and explain why. Do NOT call the cancellation tool.
3. **REFUNDABLE TICKETS:** If the policy permits a refund (and any conditions like
'24 hours prior' are met), you should proceed to call the `cancel_ticket` tool.

### TONE
Professional, objective, and direct. Do not apologize for enforcing company policy.
"""
```

```
# Set up our agent
agent_executor = create_react_agent(
    llm,
    tools,
    prompt = system_prompt)
```

```
query = "I want to cancel my Standard Ticket #9999. It is for next week."
```

```
for step in agent_executor.stream(
    {"messages": [("user", query)]},
    stream_mode="updates",
):
    print("\n--- STEP ---")
    print(step)
```

--- STEP ---

```
{'agent': {'messages': [AIMessage(content='', additional_kwargs={'tool_calls':
[{'id': 'call_KQbRFT71iPH8qfpuQtMWStW9', 'function': {'arguments':
{'query': 'Standard Ticket #9999'}, 'name': 'lookup_policy'}, 'type':
'function'}], 'refusal': None}, response_metadata={'token_usage':
{'completion_tokens': 18, 'prompt_tokens': 301, 'total_tokens': 319,
'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0,
'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details':
{'audio_tokens': 0, 'cached_tokens': 0}}, 'model_name': 'gpt-4o-mini-2024-07-18',
'system_fingerprint': 'fp_a1b343a790', 'id': 'chatcmpl-
Df7wLR7rsRBrGxc3d0J1EX0Fbg9eR', 'service_tier': 'default', 'finish_reason':
'tool_calls', 'logprobs': None}, id='run--22ffb041-a753-4e2e-9140-631370c8a31f-0',
tool_calls=[{'name': 'lookup_policy', 'args': {'query': 'Standard Ticket #9999'},
'id': 'call_KQbRFT71iPH8qfpuQtMWStW9', 'type': 'tool_call'}], usage_metadata=
{'input_tokens': 301, 'output_tokens': 18, 'total_tokens': 319,
'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details':
{'audio': 0, 'reasoning': 0}})]}}
```

--- STEP ---

```
{'tools': {'messages': [ToolMessage(content='*Note: This section is for historical
reference and applies only to bookings made before Jan 1, 2024.*\n* **All Tiers:**
Previously, all tickets were 100% refundable if cancelled within 24 hours of
booking, regardless of departure time. This has been superseded by the 2024
tiers.\n* **Work from Home:** Employees were previously required to be in the
office 5 days a week. As of 2024, we are Remote-First.\n\n* **Urgent:** Call +1-
800-VOYAGE-NOW.\n* **General:** Use the "My Trips" tab in the app.\n*
**Escalations:** Email refunds@voyage.com. Do not contact the CEO directly.\n\n*
**Refund Status:** Fully refundable up to 2 hours before departure.\n* **No-Show
```

Persistent Memory

Up to this point, each request has been treated as an independent interaction. The agent has no awareness of previous messages. In real applications, however, users ask follow-up questions that rely on earlier context. To support this, we attach a memory layer to the agent.

```
from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.memory import MemorySaver
```

```
# Create memory
memory = MemorySaver()
```

```
## Set up the agent with memory
agent = create_react_agent(
    llm,
    tools=[lookup_policy, cancel_ticket],
    checkpoint=memory
)
```

In LangGraph, memory is organized by `thread_id`. Each thread represents a separate conversation. Messages within the same thread share context. Messages in different threads remain completely isolated.

```
# set thread for user_1
config = {"configurable": {"thread_id": "user_1"}}
```

```
response = agent.invoke(
    {"messages": [("user", "I want to cancel my Saver Ticket #12345.")]},
    config=config,
)
```

```
response["messages"][-1].content
```

```
'Your Saver Ticket #12345 is non-refundable, and changes are not permitted.
Therefore, you cannot cancel this ticket for a refund. \n\nIf the airline cancels
the flight, you would be eligible for a full refund. If you fail to check in, the
ticket value will be converted to Voyage Credits valid for one year.\n\nIf you have
any further questions or need assistance, please let me know!'
```

```
config2 = {"configurable": {"thread_id": "user_2"}}

response = agent.invoke(
    {"messages": [{"user", "I want to cancel my Business class ticket for a flight tomorrow."}]},
    config=config2,
)

response["messages"][-1].content
```

'To assist you with the cancellation, I need the ticket ID for your Business class ticket. Additionally, I will check the cancellation policy to ensure that your ticket is refundable. Please provide the ticket ID.'

```
response = agent.invoke(
    {"messages": [{"user", "So I can't get a refund?"}]},
    config=config,
)
```

```
response["messages"][-1].content
```

'Correct, you cannot get a refund for your Saver Ticket #12345 as it is non-refundable. The only way to receive a refund would be if the airline cancels the flight. If you do not check in for your flight, the ticket value will be converted to Voyage Credits, which can be used for future travel. \n\nIf you have any other questions or need further assistance, feel free to ask!'

```
response = agent.invoke(
    {"messages": [{"user", "Yes, I would like to cancel anyway. My ticket id is #9999"}]},
    config=config2,
)

response["messages"][-1].content
```

'Your Business class ticket (#9999) has been successfully cancelled. If you have any further questions or need assistance with anything else, feel free to ask!'

##Fixing the lookup Policy tool

```
@tool
def lookup_policy(query: str)->str:
    """
    Consult the official Voyage Cancellation Policy
    When forming the query:
    - Always include the specific fare type mentioned by the user (e.g 'Standard
    Fare', 'Flex Fare').
    - Always include the action the customer is wanting to take e.g. cancellation,
    refund
    - Include the relevant timing conditions if mentioned
    - Do not use vague queries such as 'refund policy'

    This tool should be used before any cancellation decisions.
    """
    docs = retriever.invoke(query)
    formatted_results = []
    for doc in docs:
        section = doc.metadata.get('Section','')
        subsection=doc.metadata.get('Subsection','')
        formatted_results.append(
            """
            ---Policy Excerpt--
            Section: {section}
            Subsection"{subsection}
            {doc.page_content}
            """
        )
    return '\n\n'.join(formatted_results)
```